

Swami Sahajanand College of Computer Science

B.C.A. SEM-VI [NEP]

Subject: ADVANCE JAVA
Major (Core) - 27197

UNIT 2

APPLET AND SWING PROGRAMMING

- ◆ Life Cycle of Applet , Passing Parameters to Applet
- ◆ Event Delegation Model or Technique, Event Classes
- ◆ Introduction, Features of Swing
- ◆ J Applet, J Frame and J Panel
- ◆ Layout Managers: Flow Layout, Spring Layout, Box Layout
- ◆ J Label, J Button, J Text Field, J Check Box, J Radio Button
- ◆ J Combo Box, J List, J Menu, J Dialog

❖ What is Applet?

- An applet is a special kind of small Java program used in internet computing.
- An applet, like an application program can do anything for us.
- It is designed to be transmit over the Internet and automatically executed by a Java-compatible web browser.
- Applet is downloaded on demand, without further interaction with the user.
- If the user clicks a link that contains an applet, the applet will be automatically downloaded and run in the browser.
- Applet code executes on local computer not on the server.
- Applet code can run on with different operating systems & environment.
- All applets either directly or indirectly are subclasses of Applet.
- Applet changed Internet programming because it allows transmission of objects which can move about freely in cyberspace.

Why Applets are used? or what it can do?

- Applet are typically used to display data provided by the server, handle user input, or provide simple functions, such as a loan calculator,
- Applet provides sophisticated graphics, drawing, and image processing packages; user interface elements; networking; and event handling.

What applets can't do? [Limitation]

- Applets can't read or write to the client file system,
- Applets can't communicate with a server other than the from where applet is downloaded.
- Applets can't run any programs on the client's system.

How to pass parameter to applet?

```
<applet code=" demoprogram.class" width="300" height="80">
</applet>
```

- One can pass parameter in applet using applet tag used in html web page.
- One can retrieve passed parameter using `getParameter()` function.
- It returns the value of the specified parameter String format.
- Thus, for numeric and boolean values, one needs to convert their string into their numeric or Boolean value.
- Let us see one example. First create below html file (save it as parameterdemo.html)

```
<html>
<head>
<title>Demo applet program</title>
</head>
<body>
    <applet code=" demoprogram.class" width=300 height=80>
        <param name=fontName value=Courier>
        <param name=fontSize value=14>
        <param name=leading value=2>
        <param name=accountEnabled value=true>
    </applet>
```

</body>

</html>

- Now create .java file and write below given applet code within it. (save it as parameterdemo.java)

```
import java.awt.*;
```

```
import java.applet.*;
```

```
public class appletdemo extends Applet
```

```
{
```

```
    String fontName;
```

```
    int fontSize;
```

```
    float leading;
```

```
    boolean active;
```

```
    // Initialize the string to be displayed.
```

```
    public void start()
```

```
    {
```

```
        String param;
```

```
        fontName = getParameter("fontName");
```

```
        if(fontName == null)
```

```
            fontName = "Not Found";
```

```
            param = getParameter("fontSize");
```

```
            try
```

```
            {
```

```
                if(param != null) // if not found
```

```
                    fontSize = Integer.parseInt(param);
```

```
                else
```

```
                    fontSize = 0;
```

```
            }
```

```
            catch(NumberFormatException e)
```

```
            {
```

```
                fontSize = -1;
```

```
            }
```

```
            param = getParameter("leading");
```

```
            try
```

```
            {
```

```
                if(param != null) // if not found
```

```
                    leading = Float.valueOf(param).floatValue();
```

```
                else
```

```
                    leading = 0;
```

```
            }
```

```
            catch(NumberFormatException e)
```

```
            {
```

```
                leading = -1;
```

```
            }
```

```
        param = getParameter("accountEnabled");
        if(param != null)
            active = Boolean.valueOf(param).booleanValue();
    }
    // Display parameters.
    public void paint(Graphics g)
    {
        g.drawString("Font name: " + fontName, 0, 10);
        g.drawString("Font size: " + fontSize, 0, 26);
        g.drawString("Leading: " + leading, 0, 42);
        g.drawString("Account Active: " + active, 0, 58);
    }
}
```

Explain Delegation Event Model.

- The modern approach to handling events is based on the delegation event model.
- using listener objects, to handle event is referred to as the delegation event model
- It defines standard and consistent mechanisms to generate and process events.
- Its concept is quite simple: a source generates an event and sends it to one or more listeners.
- In this scheme, the listener simply waits until it receives an event.
- Once an event is received, the listener processes the event and then returns.
- The advantage of this design is that the application logic that processes events is cleanly separated from the user interface logic that generates those events.
- A user interface element is able to “delegate” (assign) the processing of an event to a separate piece of code.
- In the delegation event model, listeners must register with a source in order to receive an event notification.
- This provides an important benefit: notifications are sent only to listeners that want to receive them.
- Let us see this by example.
- Suppose the user clicks a button in the GUI program. The button becomes the source of this event.
- The event that is generated as a result of the mouse click is associated with the Button.
- An event always has a source object—in this case the Button object.
- When the button is clicked, it creates a new object that represents and identifies this Keystrokes, Mouse Actions.
- To link the listener object to an event source a listener interface has to connect to the source.
- One can register a listener object with a source by calling a particular method in the source object.

What are Event Classes? List them.

- The classes that represent events are called Event classes.
- It is basic part of Java’s event handling mechanism.
- These classes are used to handle various events in applet program.
- At the root of the Java event class hierarchy is EventObject, which is in java.util. It is the superclass for all events. Its one constructor is shown here:
- EventObject(Object src).

- Here, src is the object that generates this event.
- EventObject contains two methods: getSource() and toString().
- The getSource() method returns the source of the event. Its general form is shown here: Object getSource()
- toString() returns the string equivalent of the event.
- The class AWTEvent, defined within the java.awt package, is a subclass of EventObject.
- It is the superclass (either directly or indirectly) of all AWT-based events used by the delegation event model. Its getID() method can be used to determine the type of the event.
- The prototype of this method is shown here: int getID()

Event Class	Description
ActionEvent	Generated when a button is pressed, a list item is double-clicked, or a menu item is selected
AdjustmentEvent	Generated when a scroll bar is manipulated.
ComponentEvent	Generated when a component is hidden, moved, resized, or becomes visible.
ContainerEvent	Generated when a component is added to or removed from a container.
FocusEvent	Generated when a component gains or loses keyboard focus.
InputEvent	Abstract superclass for all component input event classes.
ItemEvent	Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
KeyEvent	Generated when input is received from the keyboard.
MouseEvent	Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
MouseWheelEvent	Generated when the mouse wheel is moved.
TextEvent	Generated when the value of a text area or text field is changed.
WindowEvent	Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

The ActionEvent Class

- An ActionEvent is generated when a button is pressed, a list item is double-clicked, or a menu item is selected.
- The ActionEvent class defines four integer constants that can be used to identify any modifiers associated with an action event: ALT_MASK, CTRL_MASK, META_MASK, and SHIFT_MASK.
- In addition, there is an integer constant, ACTION_PERFORMED, which can be used to identify action events.
- ActionEvent has these three constructors:
 - ActionEvent (Object src, int type, String cmd)
 - ActionEvent (Object src, int type, String cmd, int modifiers)
 - ActionEvent (Object src, int type, String cmd, long when, int modifiers)
- Here, src is a reference to the object that generated this event. The type of the event is specified by type, and its command string is cmd. The argument modifiers indicates which modifier keys

(ALT, CTRL, META, and/or SHIFT) were pressed when the event was generated. The when parameter specifies when the event occurred.

- You can obtain the command name for the invoking `ActionEvent` object by using the
- `getActionCommand()` method which return string.

The AdjustmentEvent Class

- An `AdjustmentEvent` is generated by a scroll bar. There are five types of adjustment events.
- The `AdjustmentEvent` class defines integer constants that can be used to identify them. It is given below

BLOCK_DECREMENT	The user clicked inside the scroll bar to decrease its value.
BLOCK_INCREMENT	The user clicked inside the scroll bar to increase its value.
TRACK	The slider was dragged.
UNIT_DECREMENT	The button at the end of the scroll bar was clicked to decrease its value.
UNIT_INCREMENT	The button at the end of the scroll bar was clicked to increase its value.

- `AdjustmentEvent` has only one constructor:
- `AdjustmentEvent(Adjustable src, int id, int type, int data)`
- Here, `src` is a reference to the object that generated this event. The `id` specifies the event. The type of the adjustment is specified by `type`, and its associated data is `data`. The `getAdjustable()` method returns the object that generated the event. Its has following syntax:
- `Adjustable getAdjustable()`
- The type of the adjustment event may be obtained by the `getAdjustmentType()` method.
- It returns one of the constants defined by `AdjustmentEvent`. The general form is shown here:
- `int getAdjustmentType()`
- The amount of the adjustment can be obtained from the `getValue()` method, shown here:
- `int getValue()`
- For example, when a scroll bar is manipulated, this method returns the value represented by that change.

The ComponentEvent Class

- A `ComponentEvent` is generated when the size, position, or visibility of a component is changed.
- There are four types of component events. The `ComponentEvent` class has 4 integer constants that can be used to identify them. It is given below

COMPONENT_HIDDEN	The component was hidden.
COMPONENT_MOVED	The component was moved.
COMPONENT_RESIZED	The component was resized.
COMPONENT_SHOWN	The component became visible.

- `ComponentEvent` has one constructor:
- `ComponentEvent(Component src, int type)`
- Here, `src` is a reference to the object that generated this event. The type of the event is specified by `type`.
- `ComponentEvent` is the superclass either directly or indirectly of `ContainerEvent`, `FocusEvent`, `KeyEvent`, `MouseEvent`, and `WindowEvent`.
- The `getComponent()` method returns the component that generated the event. It is given below:
- `Component getComponent()`

The ContainerEvent Class

- A ContainerEvent is generated when a component is added to or removed from a container.
- There are two types of container events. The ContainerEvent class defines int constants that can be used to identify them: it is given below

COMPONENT_ADDED	Return true if component is added
COMPONENT_REMOVED	Return false if component is removed

- ContainerEvent is a subclass of ComponentEvent and has this constructor:
- ContainerEvent(Component src, int type, Component comp)
- Here, src is a reference to the container that generated this event. The type of the event is specified by type, comp specifies the component that has been added to or removed from the container.
- You can obtain a reference to the container that generated this event by using the getContainer() method. It has following syntax:
- Container getContainer()

The FocusEvent Class

- A FocusEvent is generated when a component gains or losses input focus. These events are identified by the 2 integer constants.

FOCUS_GAINED	Return true if component get focus
FOCUS_LOST	Return false if component lost focus

- FocusEvent is a subclass of ComponentEvent and has below 3 constructors:
- FocusEvent(Component src, int type)
- FocusEvent(Component src, int type, boolean temporaryFlag)
- FocusEvent(Component src, int type, boolean temporaryFlag, Component other)
- Here, src is a reference to the component that generated this event. The type of the event is specified by type. The argument temporaryFlag is set to true if the focus event is temporary. Otherwise, it is set to false.
- The other component involved in the focus change, called the opposite component, is passed in other.
- If a FOCUS_GAINED event occurred, other will refer to the component that lost focus. Conversely, if a FOCUS_LOST event occurred, other will refer to the component that gains focus.

The InputEvent Class

- The abstract class InputEvent is a subclass of ComponentEvent and is the superclass for component input events. Its subclasses are KeyEvent and MouseEvent.
- To test if a key was pressed at the time an event is generated, use the isAltDown(), isAltGraphDown(), isControlDown(), isMetaDown(), and isShiftDown() methods. All these method returns Boolean value. If the particular key is pressed it returns true otherwise it returns false.

The ItemEvent Class

- An ItemEvent is generated when a check box or a list item is clicked or when a checkable menu item is selected or deselected. There are two types of item events, which are identified by the 2 integer constants:

DESELECTED	The user deselected an item.
SELECTED	The user selected an item.
ITEM_STATE_CHANGED	The user change state of the item

- ItemEvent has this constructor:
- ItemEvent(ItemSelectable src, int type, Object entry, int state)

- Here, src is a reference to the component that generated this event. For example, this might be a list or choice element. The type of the event is specified by type. The specific item that generated the item event is passed in entry. The current state of that item is in state.
- The getItem() method can be used to obtain a reference to the item that generated an event. Its syntax is shown here:
- Object getItem()

The KeyEvent Class

- A KeyEvent is generated when keyboard input occurs. There are three types of key events, which are identified by these integer constants

KEY_PRESSED	Return true if key is pressed
KEY_RELEASED	Return true if key is released
KEY_TYPED	Return true only when a character is generated

- There are many other integer constants that are defined by KeyEvent. For example, VK_0 through VK_9 and VK_A through VK_Z define the ASCII equivalents of the numbers and letters.
- The VK constants specify virtual key codes and are independent of any modifiers, such as control, shift, or alt.
- KeyEvent is a subclass of InputEvent. It has one below given constructors:
- KeyEvent(Component src, int type, long when, int modifiers, int code, char ch)
- Here, src is a reference to the component that generated the event. The type of the event is specified by type. The system time at which the key was pressed is passed in when. The modifiers argument indicates which modifiers were pressed when this key event occurred.
- This class also has two other important method. These are getKeyChar and getKeyCode().
- getKeyChar() return pressed character and getKeyCode() returns the key code.

The MouseEvent Class

- There are eight types of mouse events. The MouseEvent class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

- MouseEvent is a subclass of InputEvent. It has one following constructors:
- MouseEvent(Component src, int type, long when, int modifiers, int x, int y, int clicks, boolean triggersPopup)
- Here, src is a reference to the component that generated the event. The type of the event is specified by type. The system time at which the mouse event occurred is passed in when. The modifiers argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in x and y. The click count is passed in clicks. The triggersPopup flag indicates if this event causes a pop-up menu to appear on this platform.
- Few commonly used methods of this class are given below

int getX()	Return x position of mouse when button is pressed within component
int getY()	Return y position of mouse when button is pressed within component

Point getPoint()	returns a Point object that contains the X,Y coordinates in its integer members: x and y.
int getClickCount()	Return number of mouse clicks for this event

The MouseEvent Class

- TheMouseEvent class represent mouse wheel(Scroll Button) event.
- It is sub-class of mouse event class.
- Scroll buttons are basically used to scroll up or down in documents.
- MouseEvent defines these two integer constants:

WHEEL_BLOCK_SCROLL	A page-up or page-down scroll event occurred.
WHEEL_UNIT_SCROLL	A line-up or line-down scroll event occurred.

The KeyEvent Class

- The KeyEvent Class generates events when user press key either in textbox or textarea.
- It has only one constructor TEXT_VALUE_CHANGED which indicate that text value is changed in component.
- It has only one following constructor
KeyEvent(Object src, int type)
- Here, src is a reference to the object that generated this event. The type of the event is specified by type.
- This event do not return pressed character within textbox or textarea.

The WindowEvent Class

- The WindowEvent class represent event of windows of Graphical User Interface.
- There are ten types of window events.
- TheWindowEvent class defines integer constants that can be used to identify them. The constants and their meanings are shown here:

WINDOW_ACTIVATED	The window was activated.
WINDOW_CLOSED	The window has been closed.
WINDOW_CLOSING	The user requested that the window be closed.
WINDOW_DEACTIVATED	The window was deactivated.
WINDOW_DEICONIFIED	The window was deiconified.
WINDOW_GAINED_FOCUS	The window gained input focus.
WINDOW_ICONIFIED	The window was iconified.
WINDOW_LOST_FOCUS	The window lost input focus.
WINDOW_OPENED	The window was opened.
WINDOW_STATE_CHANGED	The state of the window changed.

- WindowEvent is a subclass of ComponentEvent. It defines several constructors.
- WindowEvent(Window src, int type)
- Here, src is a reference to the object that generated this event. The type of the event is type
- A commonly used method in this class is getWindow().
- It returns the Window object that generated the event. It has following syntax
- Window getWindow()

❖ **Swing Programming**

- **Swing** is a part of Java Foundation Classes (JFC) that is *used to create window-based applications*. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.
- Swing is a set of classes that provide more powerful and flexible components than AWT.
- Swing provides familiar components like buttons, checkbox and label and also adds new components like tabbed pane, scroll panes, tree and tables.
- In swing button may have both an image and a text string associated with it. the image can be changed as the state of the button changes.
- The swing related classes are contained in **javax.swing** and its sub packages.
- The Java Foundation Classes (JFC) are a set of GUI components which simplify the development of desktop applications.

❖ **Difference between AWT and Swing**

No.	AWT	Swing
1	AWT stands for Abstract windows toolkit.	Swing is also called as JFC's (Java Foundation classes).
2	AWT components are called Heavyweight component.	Swings are called light weight component because swing components sits on the top of AWT components and do the work.
3	AWT components require java.awt package.	Swing components require javax.swing package.
4	AWT components are platform dependent.	Swing components are made in purely java and they are platform independent.
5	This feature is not supported in AWT.	We can have different look and feel in Swing.
6	This feature is not available in AWT.	Swing components are called "lightweight" because they do not require a native OS object to implement their functionality components like JButton, JTextArea, etc., are lightweight because they do not have an OS peer.

7	AWT is a thin layer of code on top of the OS.	Swing is much larger. Swing also has very much richer functionality.
8	Using AWT, you have to implement a lot of things yourself.	Swing has them built in.
9	With AWT, you have 21 "peers" (one for each control and one for the dialog itself). A "peer" is a widget provided by the operating system, such as a button object or an entry field object.	With Swing, you would have only one peer, the operating system's window object. All of the buttons, entry fields, etc. are drawn by the Swing package on the drawing surface provided by the window object.

❖ **JComponents:**

1. JApplet
2. JLabel
3. JTextField
4. JButton
5. JCheckBox
6. JRadioButton
7. JComboBox
8. JtabbedPane
9. JMenus

1. JApplet :

- Fundamental of Swing is **JApplet** class, which extends Applet.
- Applets that use Swing must be subclasses of JApplet,do.
- JApplet is rich with functionality .It supports various “panes”,such as **content pane,glass pane and root pane.**
- When adding a component to an instance of JApplet,do not invoke the add() method of the applet.but call add()for content pane of the JApplet object.
- We can obtain content pane using **getContentPane()** method.
- The **add()** method of container can be used to add a component to content pane. void add (component) //component which you want to add

2. JLabel:

- Swing labels are instances of JLabel class. which extends JComponent.
- It can display text and or icon.
- **Constructor of JLabel**
 1. **JLabel(Icon i)**
 2. **JLabel(String s)**
 3. **JLabel(String s, Icon I, int align)**

Align is either LEFT, RIGHT, CENTER, LEADING or TRAILING.

- **Methods:**
 1. **Icon getIcon()**
 2. **String getText()**
 3. **void setIcon()**
 4. **void setText()**

3. JTextField:

- Swing text field is encapsulated by JTextField class, which extends JComponent.
- It provides functionality that is common to Swing text Components. One of its sub class is JTextField, which allows you to edit one line of text.
- **Constructor**
 1. **JTextField()**
 2. **JTextField(int cols)**
 3. **JTextField(String s, int cols)**
 4. **JTextField(String s)**

4. JButton:

- Swing buttons provide features that are not found in the Button class defined by the AWT.
- Swing buttons are subclasses of AbstractButton class which extends JComponent.
- AbstractButton is a super class for push buttons, checkboxes and radio buttons. It contains many methods that allow you to control the behaviour of buttons, checkboxes and radio buttons.
- Concrete subclasses of AbstractButton generate action events when they are pressed. Method used for even handling are,
 - void addActionListener(ActionListener l)**
 - void removeActionListener(ActionListener al)**

- Methods for Button
 - 1) **public void setText(String s):** is used to set specified text on button.
 - 2) **public String getText():** is used to return the text of the button.
 - 3) **public void setEnabled(boolean b):** is used to enable or disable the button.
 - 4) **public void setIcon(Icon b):** is used to set the specified Icon on the button.
 - 5) **public Icon getIcon():** is used to get the Icon of the button.
 - 6) **public void setMnemonic(int a):** is used to set the mnemonic on the button.
 - 7) **public void addActionListener(ActionListener a):** is used to add the action listener to this object.
- **JButton** class provides the functionality of a push button. JButton allows an icon, a string or both to be associated with the push button.
- Constructor for JButton:
 1. JButton(Icon i)
 2. JButton(String s)
 3. JButton(String s, Icon i) //s and i are the string and icon used for the button.

Example:

```
import
javax.swing.*; public
class ImageButton
{
    ImageButton()
    {
        JFrame f=new JFrame();
        JButton b=new JButton("Click"); b.setBounds(130,100,100, 40);
        f.add(b);
        f.setSize(300,400);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String[] args)
    {
        new ImageButton();
    }
}
```

 }

5. JCheckBox

- **JCheckBox** class which provides the functionality of a check box, is a subclass of **AbstractButton** class. Its immediate superclass is **JToggleButton**.
- It supports two states true or false.
- We can associate an icon, string, or the state with the checkbox.
- **JCheckBox constructors:**
 1. **JCheckBox(Icon c)**
 2. **JCheckBox(Icon c,boolean state)**
 3. **JCheckBox(String s)**
 4. **JCheckBox(String s,boolean state)**
 5. **JCheckBox(String s,Icon i)**
 6. **JCheckBox(String s,Icon c,boolean state)**

Here, i is the icon for the button.s is text to be displayed.If state is true,the checkbox is initially selected else not.

- The state of the check box can be changed via following method: **void setSelected(boolean state)**
- When a check box is selected or deselected ,an item event is generated.this is handled by **itemStateChanged()**.
- The **getItem()** method is used to get **JCheckBox** object that generated the event.
- The **getText()** method gets the text for that check box and text inside the text field.
- To select only one you have to create a object of **ButtonGroup** class and add all components in the object.

Example :

```
import
javax.swing.*;
import java.awt.*;
import
java.awt.event.*;
```

```
class JCheckBoxDemo extends JFrame implements ItemListener
{
    JTextField tf;
    JCheckBox jch1,jch2;
```

```

    JCheckBoxDemo()
    {
        super("Check Box Demo");
        setLayout(new FlowLayout()); tf
        = new JTextField(15); add(tf);
        jch1 = new JCheckBox("apple");
        add(jch1);
        jch1.addItemListener(this);
        jch2 = new JCheckBox("banana");
        add(jch2); jch2.addItemListener(this);
        setVisible(true);
        setSize(300,300); setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void itemStateChanged(ItemEvent ie)
    {
        JCheckBox jc=(JCheckBox)ie.getItemSelectable();
        tf.setText(jc.getText()+" is selected");
    }

    public static void main(String args[])
    {
        new JCheckBoxDemo();
    }
}

```

6. JRadioButton

- Radio buttons are supported by the **JRadioButton** class, which is a concrete implementations of **AbstractButton**. **JRadioButton**'s immediate superclass is **JToggleButton**, which provides support for two state buttons.
- Radio buttons must be configured into a group. Only one of the buttons in that group can be selected at any time.
- The **ButtonGroup** class is instantiated to create a button group. Its default constructor is invoked for this purpose. Elements are then added to the button group via **add()** method.

void add(AbstractButton ab) //ab is reference to the button to be added to group.

- **Constructor for JRadioButton**

1. **JRadioButton (Icon c)**

2. JRadioButton (Icon c,boolean state)
3. JRadioButton (String s)
4. JRadioButton (String s,boolean state)
5. JRadioButton (String s,Icon i)
6. JRadioButton (String s,Icon c,boolean state)

Example:

```
import javax.swing.*;
import java.awt.*; import
java.awt.event.*;

class JRadioButtonDemo extends JFrame implements ItemListener
{
    JTextField tf;
    JRadioButton f,m;
    ButtonGroup group;
    JRadioButtonDemo()
    {
        super("Radio Button Demo");
        setLayout(new FlowLayout()); tf = new
        JTextField(15); add(tf);
        f = new JRadioButton("Female"); add(f);
        f.addItemListener(this);
        m = new JRadioButton("Male"); add(m);
        m.addItemListener(this);
        group = new ButtonGroup(); group.add(f);
        group.add(m); setVisible(true);
        setSize(300,300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void itemStateChanged(ItemEvent ie)
    {

    }

    public static void main(String args[])
    {
        new JRadioButtonDemo();
    }
}
```

7. JComboBox

- Swing provides a combo box(a combination of a text field and a drop down list) through the **JComboBox** class, which extends **JComponent**.
- A combo box normally displays one entry. However, it can also display a drop-down list that allows a user to select a different entry.
- You can also type your selection into the text field.
- **Constructor of JComboBox**

1. JComboBox()

2. JComboBox(Vector v) //v is vector that initializes the combo box.

- Items are added to the list of choices via the **addItem()** method,

```
void addItem( Object obj)           //obj is object to be added to the
combo box import javax.swing.*;
import java.awt.*;
import
java.awt.event.*;
```

```
class JComboBoxDemo extends JFrame implements ItemListener
```

```
{
    JComboBox
    cb;
    JLabel jl;
    String lang[] = {"JAVA","DOTNET","VB","PHP","C++"};
    JComboBoxDemo()
    {
        super("Combo Box Demo");
        setLayout(new FlowLayout());
        jl=new JLabel("          "); add(jl);
        cb = new JComboBox(lang);
        add(cb); cb.addItemListener(this);
        setVisible(true); setSize(300,300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void itemStateChanged(ItemEvent ie)
    {
```

```

    jl.setText("you have selected :"+cb.getItemAt(cb.getSelectedIndex()));
}
public static void main(String args[])
{
    new JComboBoxDemo();
}
}

```

8. JMenu:

- We can create a Menu Bar which contains several menus.
- Each menu can have several menu items. The separator can also be used to separate out these menus.
- JMenuBar :- This class creates a menu bar.
- JMenu(String s) :- This class creates several menus. The string s denotes the name of the menus.
- JMenuItem(String s) :- The menu items are the parts of menus. The string s denotes the name of the menu item. setMnemonic(char c) :- The character which is passed to it as an argument becomes the mnemonic key. Hence using alt+c you can select that particular menu.
- JSeparator :- This is the constructor of the class JSeparator which adds separating line between the menu items.
- setMenuBar :- This method is used to set menu bar to a specific frame.

Example :

```

import javax.swing.*;
import
javax.swing.event.*;
import java.awt.*;
import java.awt.event.*;

class Menu extends JFrame
{
    JMenuBar menubar;
    JMenu file,color;
    JMenuItem new1,open,save,close;
    JMenuItem red,blue,green; JButton
    btnNew,btnSave,btnExit; Menu()
    {
        super("Menu Sample Demo");
        setLayout(new FlowLayout());
    }
}

```

```
ImageIcon fileicon = new ImageIcon("images/New.png"); ImageIcon
saveicon = new ImageIcon("images/save.png"); ImageIcon closeicon =
new ImageIcon("images/exit.png"); btnNew = new JButton(fileicon);
btnSave = new JButton(saveicon); btnExit =
new JButton(closeicon); menubar = new
JMenuBar();
file = new JMenu("File");
file.setMnemonic(KeyEvent.VK_F); add(file);
new1 = new JMenuItem("New",fileicon); add(new1);
file.add(new1);
save = new JMenuItem("Save",saveicon); add(save);
file.add(save);
close = new JMenuItem("Close",closeicon);
close.setMnemonic(KeyEvent.VK_X); add(close);
file.add(close);
close.addActionListener(new ActionListener()
{
public void actionPerformed(ActionEvent ae)
{
    System.exit(0);
}
});

color = new JMenu("Color");
add(color);
red = new JMenuItem("Red");
add(red);
color.add(red); menubar.add(file);
menubar.add(color);
setJMenuBar(menubar);
setVisible(true); setSize(300,300);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public static void main(String args[])
{
    new Menu();
}
}
```